

Small Language Model Training and Within-Window Caching

DASE7506 MP1 Report Draft | Final Model H1rC2
Name: Qian Jinwei Student ID: 3036804854 Date: 29 September 2026

1 Task and Results

This project trains a language model from random initialization on the supplied WikiText-2 data, using a fixed BPE-2048 vocabulary and independent causal windows of 256 tokens. The objective is to reduce bits per byte (BPB) within the inference asset, CPU time and memory limits. Two types of improvements are studied: changes to the backbone architecture and training recipe, followed by a neural cache that adjusts predictions using the observed prefix within each window.

The final model, H1rC2, is a Transformer with width 320, six layers and four attention heads. It uses RMSNorm, RoPE, XSA projection and a top-2 mixture of four SwiGLU experts. The selected 4200-step weights achieve 1.512823 validation BPB without caching. Adding the cache gives 1.493983 validation BPB on CPU in FP32 with four threads. After freezing the method, full-test CPU FP32 evaluation achieves 1.511051 BPB.

Key result	Value or status
Initial baseline validation BPB	2.061173 (1200 steps)
Final validation BPB	1.493983 (CPU FP32)
Full-test BPB	1.511051 (CPU FP32)
Inference assets and peak RAM	42.580 MiB; 1.862 GiB
Validation CPU time	28.904326 s; below 29.60 s; repeat passes

2 Data and Evaluation

The data splits, tokenizer and fixed evaluator are unchanged. The validation set contains 376,599 prediction targets and 1,148,007 UTF-8 bytes. BPB is the total negative log-likelihood in natural units divided by $\ln(2)$ times the byte count: $\text{BPB} = \text{NLL} / [\ln(2) \times \text{bytes}]$. This metric evaluates predictive probability distributions rather than the subjective quality of generated text.

All development and model selection use validation results. The model receives only the input prefix of its window, with no future-token access or state carried between windows. After freezing the method, full-test CPU FP32 evaluation yields 1.5110512081 BPB and token perplexity 23.541449 over 428,405 targets and 1,292,013 UTF-8 bytes. The checkpoint, implementation, evaluator and tokenizer SHA256 values match those in the final validation output.

3 Architecture and Training

3.1 From the Baseline to the Final Backbone

The supplied baseline is a GPT with width 128, four layers, four heads and 1,088,256 parameters. It uses LayerNorm, learned positional embeddings, GELU feed-forward layers and tied input/output embeddings. The final backbone has 10,497,600 parameters and uses RMSNorm [1], RoPE [2] and SwiGLU [3], while retaining tied embeddings. Both architecture and optimization settings were selected on validation data; the combined improvement does not establish the independent benefit of every component.

Each MoE layer contains four SwiGLU experts with hidden width 320. A linear router followed by softmax selects the two highest-probability experts, renormalizes their selected probabilities and combines their outputs. Each expert computes $\text{down}[\text{SiLU}(\text{gate}(x)) \odot \text{up}(x)]$. A load-balancing auxiliary loss is added to the language-model cross-entropy to discourage reliance on only a few experts. Balanced average usage alone does not demonstrate meaningful expert specialization.

In this implementation, XSA removes the component of the attention output parallel to the current-position value vector. For normalized value vector u and attention output a , the result is $a - (a \cdot u)u$, followed by the output projection and residual connection. This definition refers specifically to the project code and is not a claim that XSA is a generally validated method. The final model does not enable BigramHash, Smear or residual-scaling initialization.

3.2 Final Training Recipe

Setting	Final value
Backbone and experts	width=320, depth=6, heads=4; 4×320, top-2
Training budget	5000 steps, batch=32, context=256
Optimizer	AdamW; LR=0.001; betas=(0.9, 0.999)
LR schedule	warmup=100; cosine horizon=5000; minimum ratio=0.1
Regularization and averaging	Dropout=0.1; WD=0.3; EMA=0.99
Routing auxiliary loss	Coefficient 0.01; gradient clipping norm 1.0
Seed and validation	seed=17; validate every 300 steps and at the end
Environment	RTX 5070; PyTorch 2.7.1+cu128; BF16 training

The full run processes 40,960,000 training targets. Validation selects the EMA weights at step 4200, corresponding to 34,406,400 targets; both budgets are reported separately. The last checkpoint has validation BPB 1.515171, so the better intermediate checkpoint is retained for inference. Formal re-evaluations use FP32.

4 Fair Comparison and Learning Curves

The initial baseline achieves a measured 2.061173 BPB after 1200 steps. The main comparison uses an additional 5000-step baseline run and the final H1r architecture. Both use batch size 32, context length 256, seed 17 and a 5000-step learning-rate schedule. Each full run processes 40,960,000 training targets, and the best checkpoint is selected using validation results.

Model	Run training targets	Best step	Validation BPB
Initial baseline	9,830,400	1200	2.061173
Baseline 5000	40,960,000	5000	1.742927
H1r without cache	40,960,000	4200	1.512823
Final H1rC2	Same as H1r; no extra training	4200	1.493983

Under the same 5000-step run budget, H1r reduces BPB by 0.230104 (13.20%) relative to the baseline. H1rC2 reaches 1.493983, a reduction of 0.248945 (14.28%). H1r peaks at step 4200, after 34,406,400 targets, while the baseline peaks at step 5000. This compares model selection under equal run budgets, not identical training exposure of the selected weights or equal compute. Architecture, capacity and training settings change together, so the gain cannot be attributed solely to MoE. The baseline score comes from a BF16 training log, whereas the final model was re-evaluated in FP32; a uniform FP32 baseline evaluation remains to be recorded.

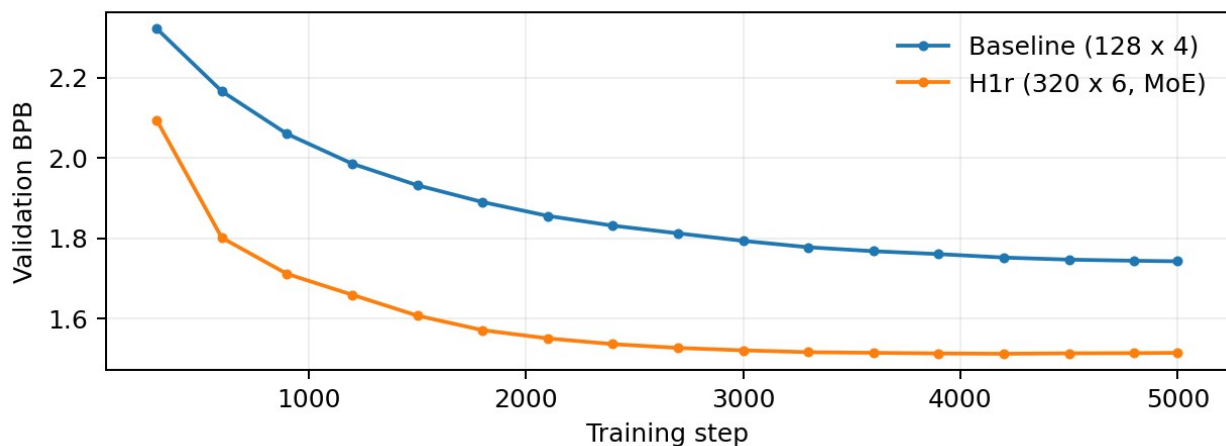


Figure 1. Validation curves under the same 5000-step schedule. The baseline reaches 1.742927 at step 5000; H1r reaches 1.512823 at step 4200 before a slight increase. Final-step points come from last_validation.

The H1r rerun achieves 1.512823 versus 1.512831 for H1, showing close reproduction with the same seed. An earlier 3600-step recipe achieves 1.530084 with seed 17 and 1.532191 with seed 42. This is limited evidence about seed variation, not a multi-seed confidence interval for the final H1rC2 model.

5 Within-Window Cache and Ablation

A continuous neural cache assigns probability to previously observed tokens according to the similarity between current and historical hidden states [4]. Here, the memory is restricted to the current independent window. No state persists across windows, and the trained backbone weights remain fixed.

When predicting the token after position t , let h_t be the final-layer representation and q_t its L2-normalized vector. For every $i < t$, compute $s_{ti} = \theta(q_t \cdot q_i)$ and apply softmax over valid historical positions to obtain a_{ti} . Aggregate by token to form $p_{\text{cache}}(w|t) = \sum_{i < t} a_{ti} 1[x_{i+1}=w]$. Since $i < t$, the latest possible stored token is x_t , which is already in the input prefix. The target x_{t+1} is not accessed.

The final distribution is $p_{\text{final}} = (1-\lambda)p_{\text{model}} + \lambda p_{\text{cache}}$. At the first position, where no history exists, the model returns p_{model} directly. Caches are constructed independently for every window and batch item. Probabilities are mixed before taking the logarithm for the evaluator. The similarity scale θ sharpens attention toward the most similar historical positions as it increases. The mechanism adds no trainable parameters and requires no external count table.

Same H1r step-4200 weights	λ	θ	Validation BPB
Cache off (CUDA FP32)	0	—	1.512823
H1rC (CUDA FP32)	0.05	10	1.494928
H1rC2 (CPU FP32)	0.04	12	1.493983

The controlled CUDA FP32 cache on/off comparison reduces BPB by 0.017895. Backbone weights and training exposure are identical, isolating the inference-time cache effect. The last row reports the final CPU re-evaluation. Its roughly 0.000945 improvement over the preceding row cannot be attributed entirely to tuning λ and θ because the device also changes. Final cache settings were selected on validation data and then frozen.

Trigram Comparison

Another inference-time candidate uses a sparse trigram table built only from training text. It retains trigrams with $\text{count} \geq 2$, uses smoothing and lower-order backoff, and mixes its probabilities with the model. On the frozen M5 backbone, BPB is approximately 1.53009 at $\lambda=0$ and 1.51859 at $\lambda=0.10$. Neural caching on the same backbone reaches approximately 1.50758. The trigram table also requires about 7.85 MiB, so it is excluded from the final method. These sweep values come from the experimenter's summary and should be archived with the original sweep JSON.

The cache gains are consistent with recent repetitions of names, fragments or patterns complementing the backbone. However, no token-level category analysis was performed, so the results do not establish that any particular linguistic phenomenon explains all of the gain.

6 Unsuccessful Variants and Limitations

Candidate screening uses successive paired experiments, including unsuccessful results. The table reports the best validation scores of models without caching under 5000-step schedules. Unless noted, the backbone uses width 320, six layers, four heads and seed 17. Runs have the same maximum training budget, but their selected checkpoints can occur at different steps.

Variant	Best BPB	Best step	Observation
4×320 top-2, drop=0.1	1.512823	4200	Retained
4×320 top-2, drop=0.125	1.513480	5000	Small difference
Hash, drop=0.1	1.584323	3300	Clearly worse
Hash, drop=0.125	1.578032	3900	Still worse
6×256 top-2	1.517904	3900	Does not beat H1r
12×128 top-2	1.519419	3900	Does not beat H1r
4×384 top-2	1.530092	3900	No gain from width
4×320 top-2, aux=0.003	1.534183	4200	No gain from lower aux
12×128 top-5, aux=0.01	1.548835	3900	No gain from higher top-k

BigramHash adds a hashed embedding of adjacent input tokens to the input representation. With 8192 buckets and embedding width 64, it adds 544,768 parameters. Although some training losses decrease, validation performance worsens, consistent with overfitting to local combinations. This mechanism differs from both a training-set trigram probability table and a temporary neural cache. Its failure does not show that all hashed memory mechanisms are ineffective.

The expert-count experiments also change expert width. For 4×320, 6×256 and 12×128 with top-2 routing, the summed active hidden widths are 640, 512 and 256. Thus, expert count is not isolated. The 12×128 top-5 variant restores this sum to 640, but routing and runtime costs still differ. The auxiliary-loss implementation uses expert selection fractions whose sum scales with top-k. A fixed coefficient of 0.01 therefore does not imply a fixed normalized auxiliary strength, and the top-5 result cannot be attributed solely to the number of active experts.

Most experiments use seed 17. Repeated validation-based selection of architecture, hyperparameters and checkpoints introduces selection bias. Machine load can also affect timing. Matching model-code hashes does not replace versioning the training script, environment and dependencies. Conclusions apply only to the tested recipes; no globally optimal architecture is claimed.

7 Resource Use and Constraint Checks

H1rC2 achieves 1.4939828818 validation BPB on CPU in FP32 with four threads. A subsequent timing measurement is 28.904326 seconds, or 4.882 times the script's 5.92-second baseline. This is below the 29.60-second threshold, so that repeat measurement passes the time limit.

Resource	Reported measurement	Limit or basis	Status
CPU scoring time	28.904326 s	5× baseline	Repeat passes
Script baseline	5.92 s	Same environment and protocol	Script reference
Time ratio and threshold	4.882×; 29.60 s threshold	≤5×	0.696 s below limit
Peak working set	1.862453 GiB	≤4 GiB	Passes
Uncompressed assets	42.579773 MiB	≤64 MiB	Passes script count

Runtime varies between runs. An earlier resource measurement was 33.204336 seconds, while direct `evaluate.py` execution recorded 24.130933 seconds. The later 28.904326-second measurement passes with about 0.696 seconds to spare. This later value was supplied by the experimenter; the raw output should accompany the reproduction materials. Comparisons should use the same machine, thread count, split and timing procedure, retaining repeated measurements.

GPU allocator fields are zero during CPU evaluation and do not measure CPU memory. The table uses the process peak working set from the resource wrapper. Reported assets total 44,648,128 bytes; the final count should include every required inference file. These checks use validation. Direct full-test scoring takes 35.750489 seconds. Without a same-environment test baseline time, the validation threshold of 29.60 seconds cannot determine whether test runtime passes.

Training and Search Costs

The final H1r rerun records `train_seconds=410.845` and `process_seconds=448.577` on an RTX 5070. The training-time field excludes intermediate validation time tracked by the script, but is not an independent measurement of GPU kernel time. The 5000-step run produces the selected 4200-step checkpoint; subsequent cache tuning does not update the backbone.

The previously archived 57 runs, deduplicated by checkpoint SHA256, total approximately 19,022 training seconds. Adding the 37.980-second baseline run gives 58 records and approximately 19,060 seconds (5.29 hours). This sum of recorded `train_seconds` is neither GPU utilization integrated over time nor sequential wall-clock duration. Unarchived failed runs, cache/trigram searches, table construction, CPU measurements and manual development are not fully included.

Conclusion

Under the same 5000-step run budget, the final H1r architecture and training recipe outperform the baseline. Within-window caching further improves predictions with fixed weights. Final CPU FP32 BPB is 1.493983 on validation and 1.511051 on the full test set after freezing the method. The later validation CPU measurement of 28.904326 seconds is below the 29.60-second threshold; reported memory and assets are also within budget. These resource checks are based on validation measurements.

8 Reproducibility and References

Repository: <https://github.com/qutcal-alt/slm-challenge>

Model weights: [H1rC2 checkpoint.pt](https://huggingface.co/H1rC2/checkpoint.pt)(GitHub)

These links point to main. The SHA256 values below identify the weight and implementation files.

Final checkpoint path relative to the code directory:

`runs/expH1rC2-w320-d6-h4-moe4x320-drop01-4200-cache004-theta12/checkpoint.pt`

The source is H1r's step-4200 EMA checkpoint. Export adds `cache_mix=0.04` and `cache_theta=12.0` without changing backbone weights. The public bundle should include `student.py` and its imported dependency `ngram.py`. Although the final `ngram_mix` is zero, the module-level import must still be satisfied.

SHA256 values fingerprint file contents for version checking. The entries identify the weights, model implementation and evaluator, respectively.

checkpoint SHA256

`cc3b4bfc717e23ccb0322e37e1b4696631edb529b4928598f67849713e9cb96d`

implementation SHA256

`3420ac2af194108d5910ae08d77dae3139ef0070ec6afe69486a0aef3a5182bf`

evaluator SHA256

`128bcb2dab0be0d427505bddb4671e3ab3a8f78e114be79a689c0f9029af133d`

Run from the code directory, replacing CHECKPOINT with the path above:

`python evaluate.py --checkpoint CHECKPOINT --device cpu --precision fp32 --split validation --threads 4`

For test reproduction, replace `--split validation` with `--split test`.

AI Assistance

ChatGPT/Codex assisted with method explanations, static code review, log analysis and report preparation. The author ran the training and evaluation and supplied the outputs. The AI environment did not rerun full PyTorch training or evaluate the final checkpoint; reported model scores come from the supplied metrics and evaluation outputs.

References and Experiment Records

[1] Zhang, B. and Sennrich, R. Root Mean Square Layer Normalization. 2019. <https://arxiv.org/abs/1910.07467>

[2] Su, J. et al. RoFormer: Enhanced Transformer with Rotary Position Embedding. 2021. <https://arxiv.org/abs/2104.09864>

[3] Shazeer, N. GLU Variants Improve Transformer. 2020. <https://arxiv.org/abs/2002.05202>

[4] Grave, E., Joulin, A. and Usunier, N. Improving Neural Language Models with a Continuous Cache. ICLR 2017. <https://arxiv.org/abs/1612.04426>

[5] DASE7506 MP1 GUIDE, supplied `model.py` and fixed `evaluate.py`. Initial baseline: `metrics.json`. The 5000-step baseline: the later `metrics.json`, archived as `01-metrics.json` (checkpoint SHA256 prefix `0a48a72cf95e`). Final backbone: `metrics(20260929-012038).json`. Cache and resource results use CPU outputs and the experimenter's subsequent timing measurement.